* Unit -.4 *

Vertu Transport Layer

Transport Layer :-

~~And to And~~ end - to - end communication.

Transport Layer provides communication services between application processes on different hosts.

Position in OSI model

Application layer
Session layer
Transport layer ✓
Network layer
Data link layer
Physical layer

Ex Sender - message is divided into segements
Transport layer adds port numbers
~~Rec~~ Receivers - reassembles data.

This process ensures correct delivery

Working

Sender side

1. Takes data from application
2. Breaks into segments
3. Adds header (Port numbers)
4. Sends to network layer

Receiver side

1. Receives ~~org~~ segements
2. Reassemble data, uses port numbers
3. sends to correct application

* Transport layer service :-

(1) process to process communication

Ensures data reaches correct application.

Uses port number (16-bit)

↳ Browser → Port 80

Gmail → port 25

(2) Multiplexing & Demultiplexing

Multiple application shares network

Sender → multiplexing

Receiver → Demultiplexing. (collect data)

(3) Flow control

- Controls speed of transmission.
- data transmitted at a rate that is accept table for both sender & receiver by managing data flow.
- Uses sliding window.

(4)

(4) Data integrity

Transport layer ensures

- Detect errors and currupted packet
- Retransmit lost packets (Track lost packet)
- Remove duplicates
- Maintain order.

(5) Congestion Control

- prevents network overload.
- Congestion → load on network > load capacity.
- Transport layer adjusts sending rate.

* Elements of Transport Protocols :-

There are the basic components / functions required to design a transport protocol

In simple words – what things a transport layer must handle to ensure reliable communication

main elements–

1. Addressing (Port Addressing)
2. Connection establishment
3. Connection release
4. Flow Control
5. Buffering
6. Error Control
7. Congestion Control
8. Multiplexing & Demultiplexing

(1) Addressing (Port Address)

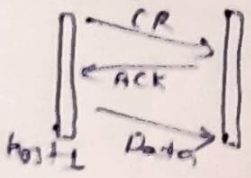
HTTP - 80
HTTPS - 443
FTP → 21

need - many apps run on same device.
port ensures correct delivery

(2) Connection Establishment -

Before sending data, connection is established between sender & receiver using three way handshake

- Ensures both sides are ready
- Synchronization.



(3) Connection Control -

After data transfer → Connection must be Closed.

- Free resources
- Avoid unnecessary traffic

4. Flow control
   - control data transmission rate
   - Sliding window Protocol
   - Prevents reciever overload.

5. Buffering
   - Temprory storage of data
   Types → Sender buffer
           → Receiver buffer.
   - Handles speed mis-match
   - Stores out-of-order packet

6. Multiplexing & Demultiplexing
   Concept - Multiplexing => Combine data from multiple apps.
   - Demutiplexing => deliver to correct app.
   - Multiple apps using internet at same time.

7. Error Control
   - Ensures reliable data transfer
   - Methods → Check sum
              → Retransmission
              → ACK/NAK

8. Congestion Control
   - Controls network overload
   - reduce sendings rate during congestion

* UDP (User Datagram Protocol) :-

- UDP is connectionless & unreliable transport layer protocol.
- UDP provides non-sequenced, fast data transmission
- It send fast data without worrying about reliability.

Characteristics -

(1) No connection setup befor sending data means it is connectionless., send data directly.

(2) Unreliable ⇒ No gaarantee of
- delivery
- Order
- Duplication.

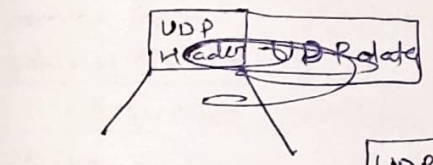
(3) Fast → No error recovery

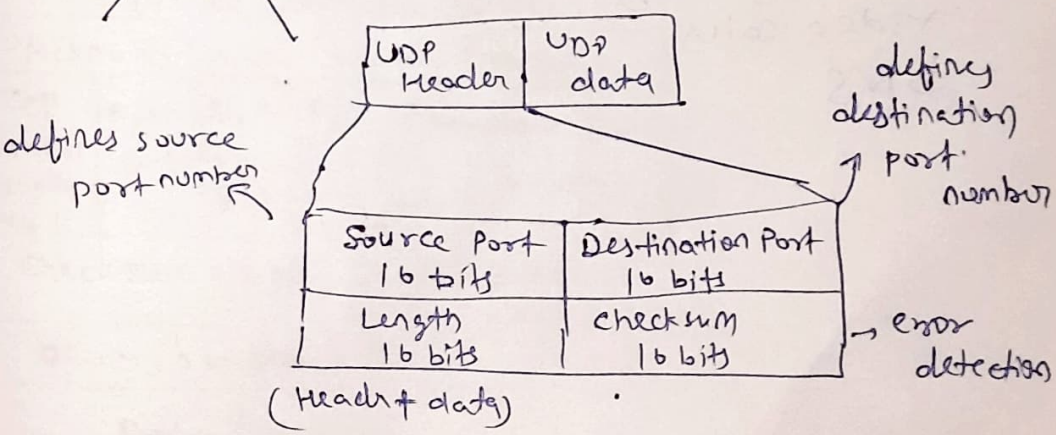
(4) Light weight → small header, less overhead.

* UDP Packet

also known as User Datagram.

UDP header size = 8 bytes





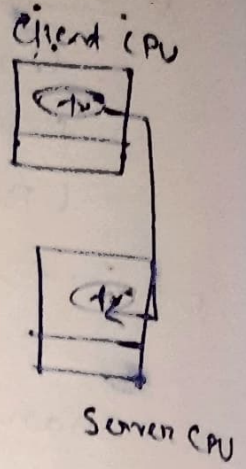
Working of UDP:-

Sender side

1. Data comes from application
2. UDP adds header
3. sends directly

Receiver side

1. Receives datagram
2. Removes header
3. Sends to application.



UDP Service

- Process to process communication
- Connectionless services
- Fast delivery
- Checksum support

Advantage

1. Fast transmission
2. Low overhead
3. Simple
4. Suitable for real-time

Disadvantage

1. No sequencing
2. No retransmission
3. Cannot identify lost packet
4. Less reliable.

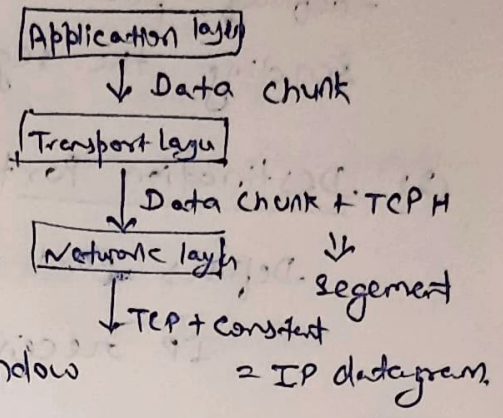
Application

- Live streaming
- Online gaming
- Video calls
- DNS

TCP (Transmission Control Protocol)

TCP provides reliable logical communication between processes.

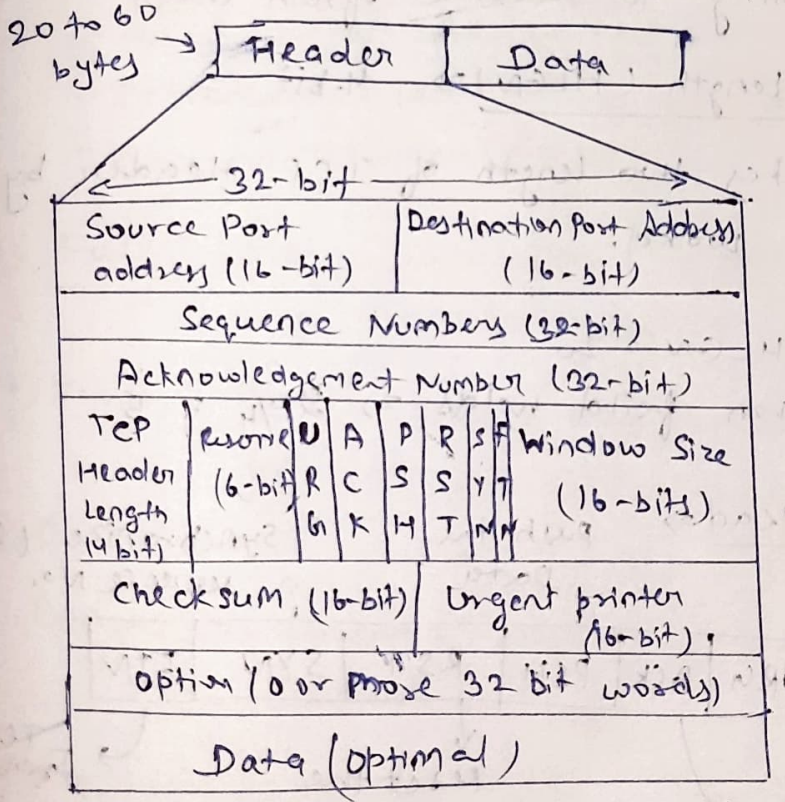
→ primary purpose to provide a reliable logical connection between pair of process.



→ Reliability ensured by
- connection oriented service
- Flow control using sliding window protocol
- Error detection using checksum.

→ Data from application layer which devided into segments.
- each segment sent via network layer.

• TCP Header & Segement Structure



(TCP Header)

Header Size = 20-60 bytes.

Fields

(1) Source Port => 16 bits
- Sender Application
- defines port no. of Application in host that IP sending the segement.

(2) Destination Port => 16 bits
Defines port no. of Application in host that IP receiving segement.

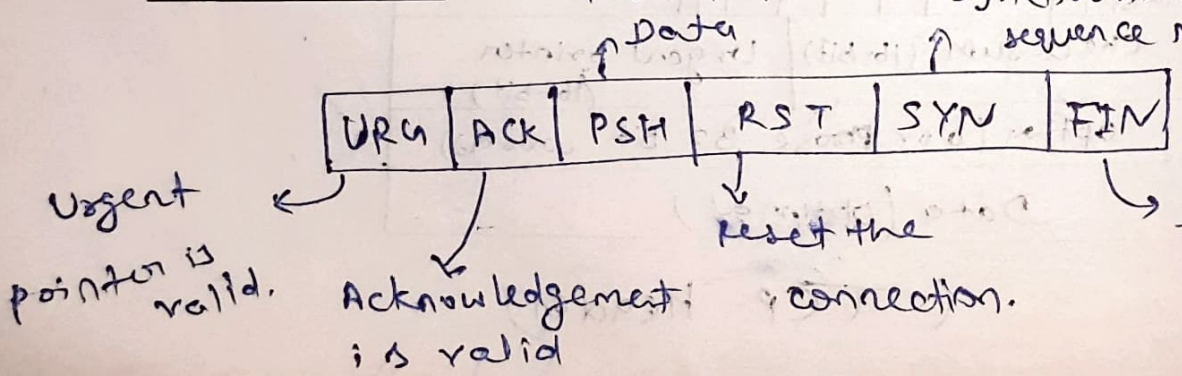
(3) Sequence Number => 32 bit
defines number assigned to first byte of data contained in segement.

(4) Acknowledgement Number => 32-bit
Defines byte number that receiver of segement is expecting to receive from other.

(5) Header Length (HLEN) => 4-bit
Indicates two length of TCP Header by numbr of 4-byte word.

If HLEN = 20
then feild holds => 20/4 = 5.

(6) Control Head =>



7. Window Size - window size of the sending TCP in Bytes. Used for flow control

8. Checksum - It holds checksum, used for error detection.

9. Urgent pointer - points to urgent data. that needs to reach the recevring process

10. Option - up to 40 bytes available for optional information in TCP Header.

11. Data field - Contains actual message to be sent.

* Features of TCP :-

1) Connection oriented → Connection must be established before communication

2) Reliable Communication
   - Uses acknowledgement
   - Retransmit lost data
   - Ensures correct delivery

3) Full Duplex - Data flows in both directions simultaneously.

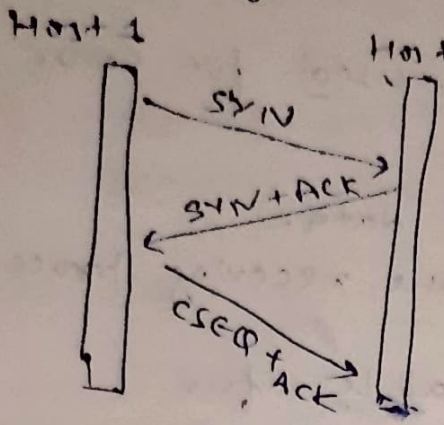
4) Flow control - Uses sliding window that prevents receiver overload.

5) Error Control - uses checksum / ACK / Retransmission

6) Congestion Control - Adjust sending rate when network is busy.

* TCP Connection Establishment

* (Using Three way handshake)

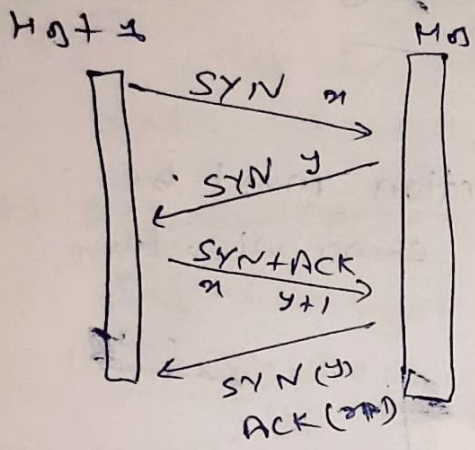


Step 1 Client send the first SYN segement

Step 2 Server sends (2nd) segement SYN + ACK

Step 3 Client send third segement just an ACK segement ACK

[Normal Core]



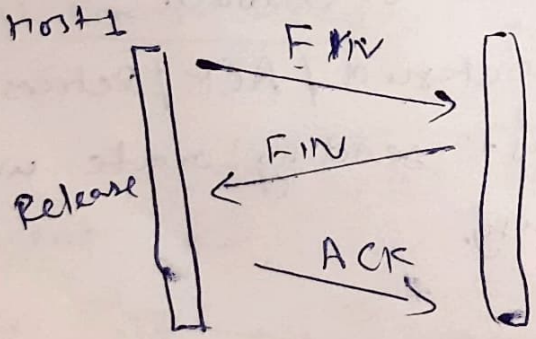
→ Both wants to establish connection.

→ Two host simultaneously attempt to establish

Collision Core

TCP Connection Release :- (Termination)

- Similar to connection release of transport layer
- Implemented using 3-way Handshake.
  - FIN used for disconnection request.
  - Connection should be released from both sides.



Release → after ack, it shut down for new data.

TCP Data transfer -

1. Data devided into segements
2. each segement has sequence number
3. Reciver sends ACK
4. Lost packet → retransmitted.

So the states are:-

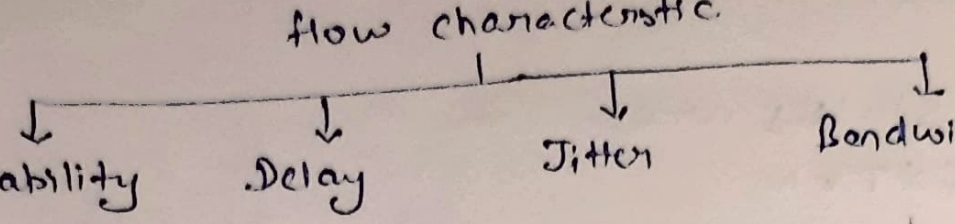
LISTEN
↳ SYN-SENT
↳ ESTABLISHED
↳ FIN-WAIT
↳ TIME-WAIT
↳ CLOSED

* Difference between TCP & UDP

Description	UDP	TCP
1. General	Simple, High speed, low functionality protocol	Full features, reliable Data transfer
2. Connection setup	Connectionless	Connection oriented
3. Data interface to application	Manage baced data is sent	Continous stream baced data is sent.
4. Reliability	Unreliable (ACK x)	Reliable (ACK ✓)
5. Retransmission	Not performed	If data lost → Retransmission Done
6. Flow control	No	Sliding window → flow control
7. error control	No	Yes
8. Overhead (extra feature)	Very low	Low but Higher then UDP
9. Transmission speed	Very High speed	High, Lower then UDP
10. Applications that use protocol	DNS, BOOTP, TFTP DHCP, SNMP multipredia Application	FTP, TELNET, SMTP, DNS, HTTP

* Quality of Service :-

In QoS we try to create a appropriate environment for the traffic.



(a) **Reliablity** — No data loss
error detection + retransmission

(b) **Delay (Latency)**

Time taken to deliver data (source to destination)

Source packet (0 time) ⟶ destination (4 times) Read (4 time)

Ex in audio conferencing, video conferencing, Remote login need minimum delay.

(c) **Jitter** Variation in delay

It is important for real time applications

Ex: Four packet ⟶ arrived

time 0, 1, 2, 3 ⟶ 20, 21, 22, 23

Same delay = 20 time. ch.

if four packet ⟶ arrived

time 0, 1, 2, 3 ⟶ 21, 23, 21, 28

delay = 21, 22, 19, 24

more relaible for priority.

(4) Bandwidth :-

Different application use different bandwidth.

Eg In video conferencing, need to send millions of bit per ~~bit~~ second to refresh a color screen, where as an email may not require that much bit rate.

* QoS Improvement Techniques

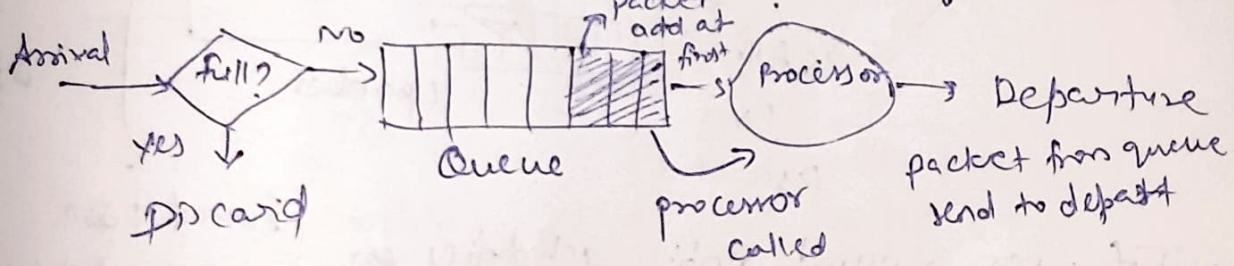
* Scheduling when packets arrive at a router, scheduling decides which packet should be sent first.

need → multiple packets arrive at same time
Router cannot send all at once
so we require proper order, fairness & priority handling.

Types (1) FIFO Queuing (First in First out)

packet is processed in same order as arrival

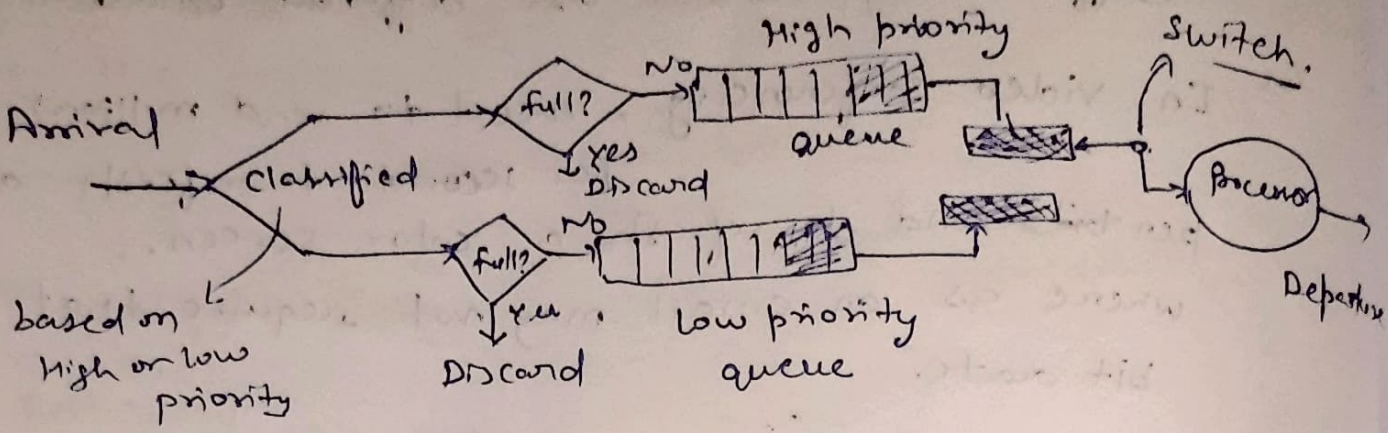
working packet arrives → stored in queue
first packet → Processed first



→ No priority & Delay for imp packets
→ Simple & easy to implement.

(2) Priority Queuing –

packet are assigned priority level.

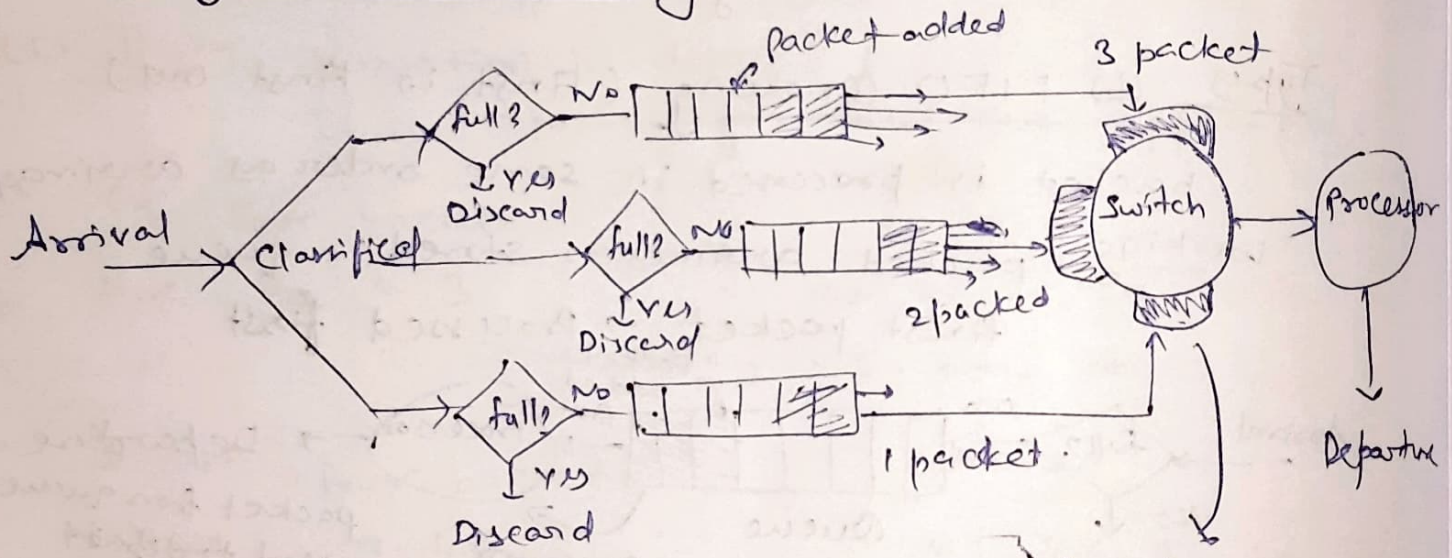


High priority processed first then switch turn of and low priority processed.

⇒ voice call → High
Email → Low

→ Imp packets sent first & low delay for critical data
→ low priority may ↓ never get chance.

(3) Weighted Fair Queuing –



→ It uses round robin scheduling for algorithm for scheduling.
→ each queue gets bandwidth based on weight.
→ fair + priority Combined / No starvation but Complex.

* Traffic Shapping :-

Technique used to -
- control the rate of data transmission
- smooth network traffic
- Prevent congestion.

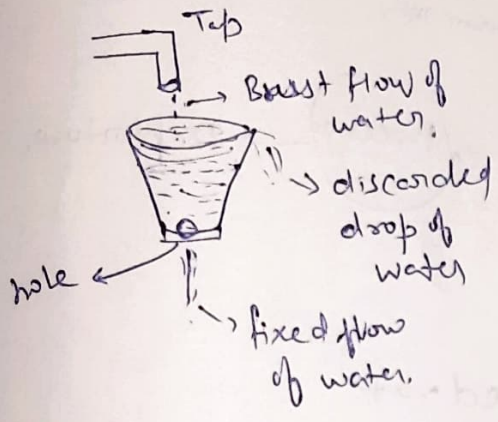
It regulates the flow of packets to improve QoS

without traffic shapping
Sudden burst of data. & packet loss.

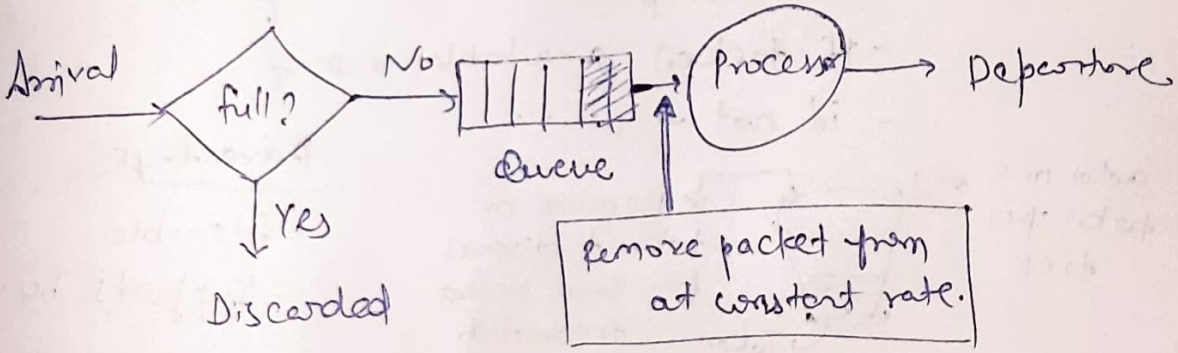
Types of Traffic shapping -
- Leaky bucket Algorithm

(1) Leaky bucket Algorithm :-

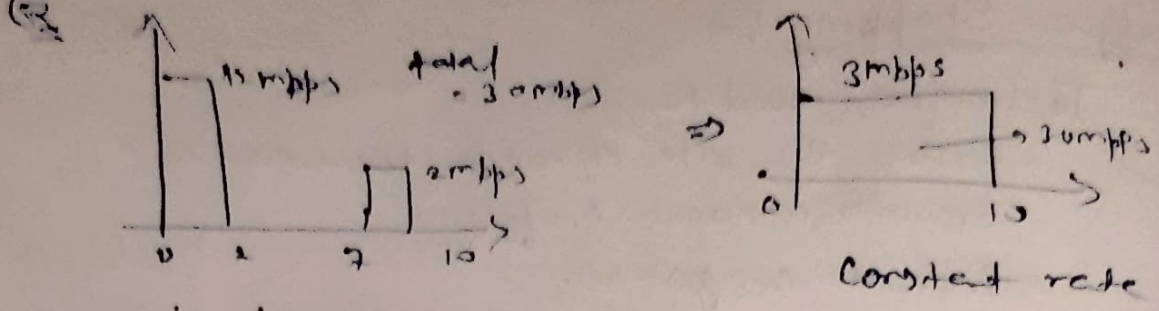
→ Data flows out at a constant rate, like water leaking from a bucket.



working
- Bucket has fix size
- Data enters randomly
- Output is constant



Algo Bursty traffic into fixed-rate traffic by averaging the data rate.



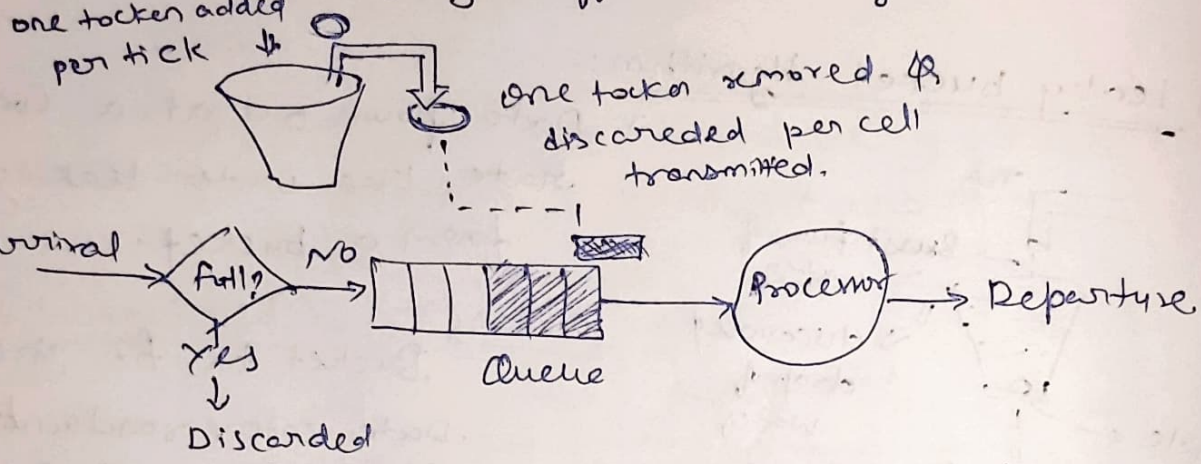
Advantage
- Smooth traffic
- Prevents congestion

Disadvantage
- Burst not allowed
- Extra packets dropped.

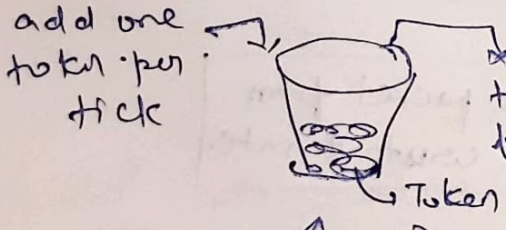
(2) Token Bucket Algorithm

It is useful when large burst traffic arrive.

It allows bursty traffic at a regulated max. rate.

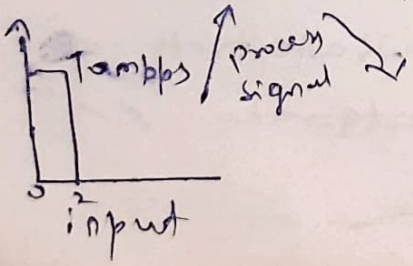


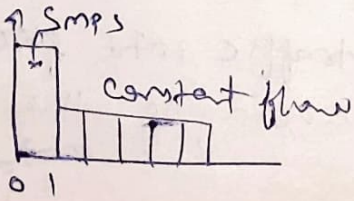
working
- Token added at fixed rate
- To send packet → token required
- If token available → send
- if not → wait.



Advantage
- flexible
- Supports burst traffic

Disadvantage
Slightly complex.





Difference between Leaky Bucket & Tocken Bucket

Feature	Leaky Bucket	Tocken Bucket
Concept	Fixed output rate	Tocken-based transmission
Output rate	Constent	Variable
Burst traffic	Not allowed	Allowed
Flexibility	Low	High
Data handling	Extra packets dropped	Tockens stored for later use
Efficiency	Lower	Higher
Traffic Type	Smooth uniform flow	Burst + smooth flow
Complexity	Simple	Slightly Compleced